

Agentes SWE

separá el que construye del que juzga

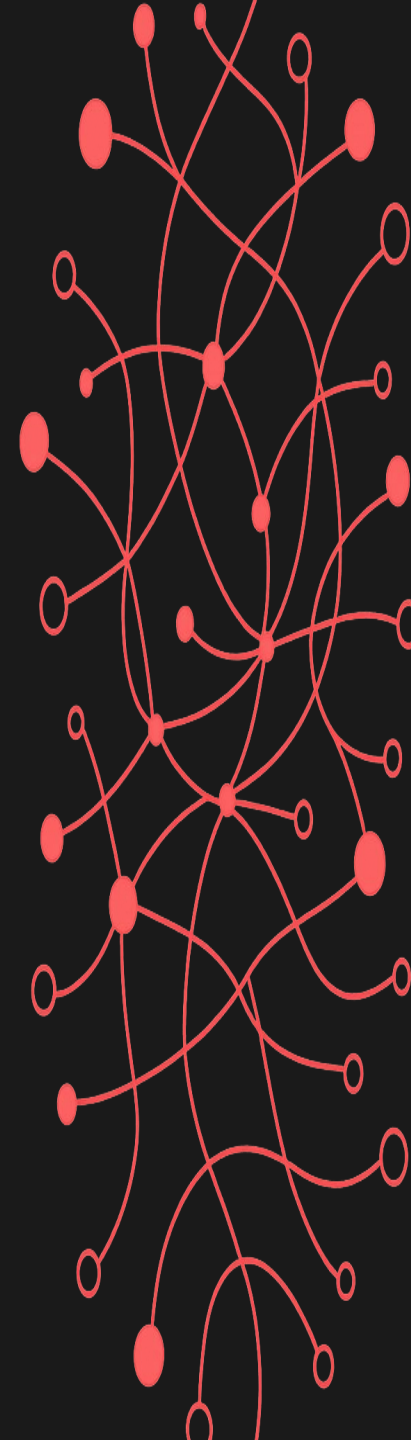
cómo un harness Generator / Evaluator construye software que funciona

1 Por qué es difícil

2 La idea clave

3 Tres builds

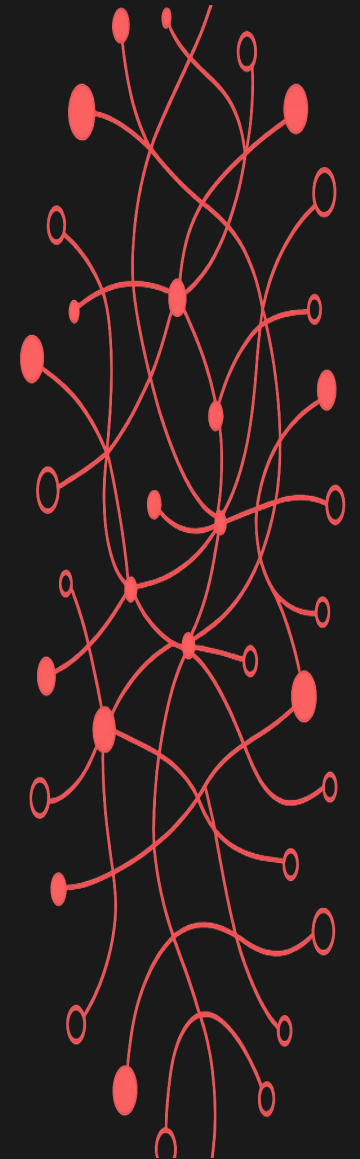
4 El harness es temporal



01

Por qué es difícil

el problema de un agente que se corrige a sí mismo



Cuando un agente corrige su propio trabajo



Ansiedad de contexto

El agente siente que tiene poco espacio en la ventana y cierra antes de tiempo.



Sesgo de autoevaluación

El agente elogia con seguridad su propio trabajo mediocre.

**Imaginá un proyecto de software con ingenieros
trabajando por turnos, donde cada nuevo
ingeniero llega sin memoria de lo que pasó en el
turno anterior.**

— Anthropic engineering blog

Lo obvio primero

Un solo agente.

Autonomía total.

Prompt:

"Construí un creador de juegos retro 2D."

20 min

hasta terminar

\$9

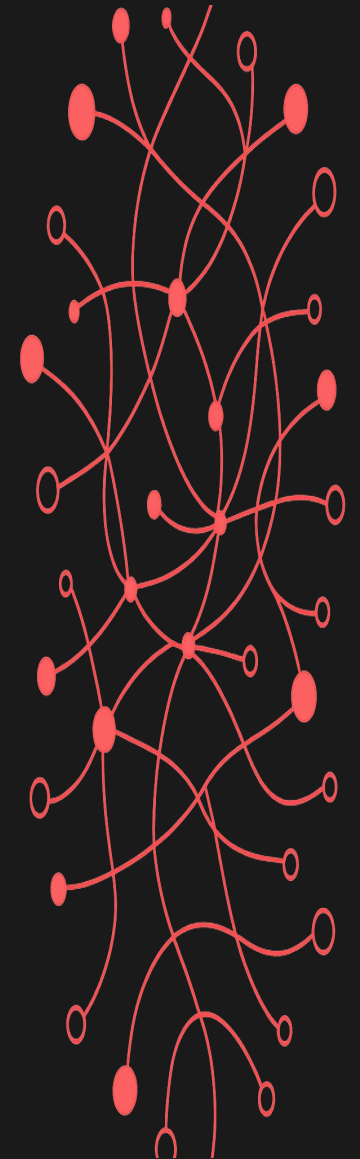
costo total

- La landing se ve pulida
- El modo de juego renderiza, pero no responde a inputs
- El agente reportó éxito.

módulo 2 de 4

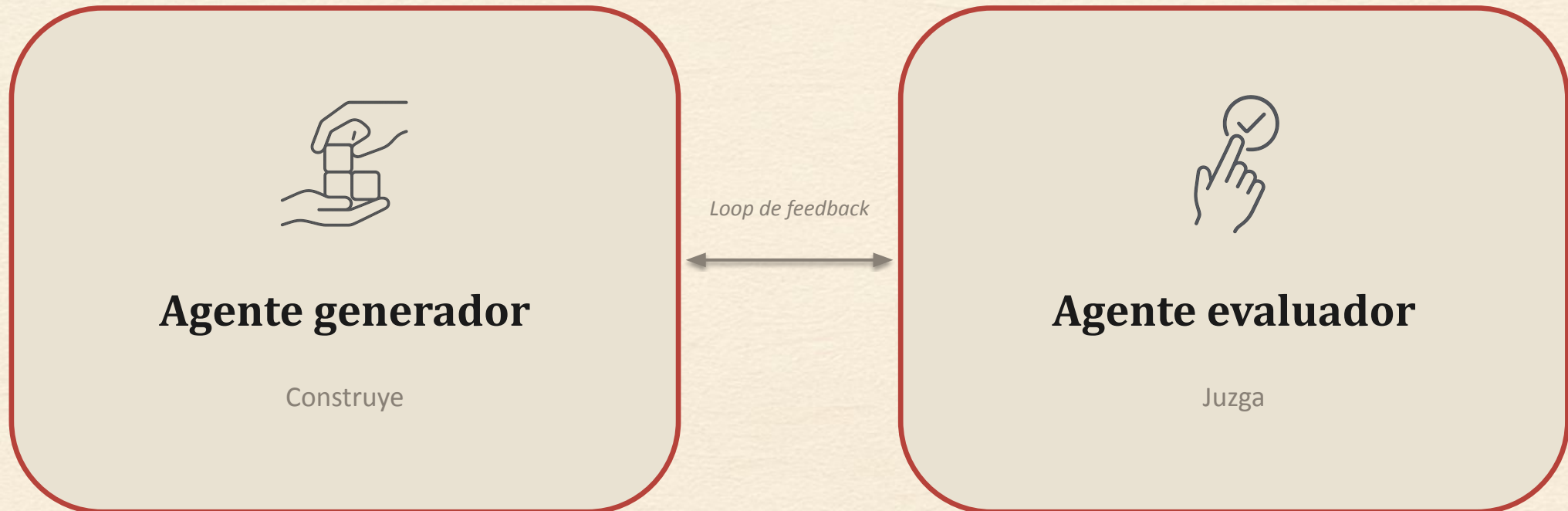
02 La idea clave

uno construye, otro juzga



Uno construye. Otro completamente separado juzga.

Se empujan mutuamente.



El harness: tres roles, un loop de feedback



El Sprint Contract es una spec verificable por máquina: el generator escribe contra ella, el evaluador la califica.

Vago

Prompt:

"Construí un editor de niveles."

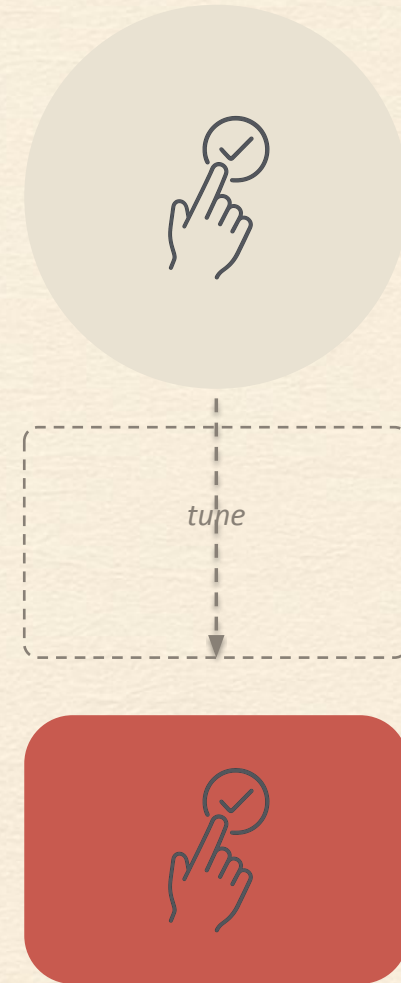
Contrato:

- **Herramienta de relleno de rectángulo**
rellena el área con click-drag
- **Verificado por:**
click en (10,10), arrastrar a (50,50), confirmar que todos los tiles del rect quedan rellenos

Un prompt vago colapsa en un contrato preciso y testeable.

Out of the box, Claude no siempre es un buen agente de QA.

Identificaba un problema real y después se auto-convencía de aprobarlo igual. El evaluator que atrapó esos bugs se tuneó durante varias rondas.





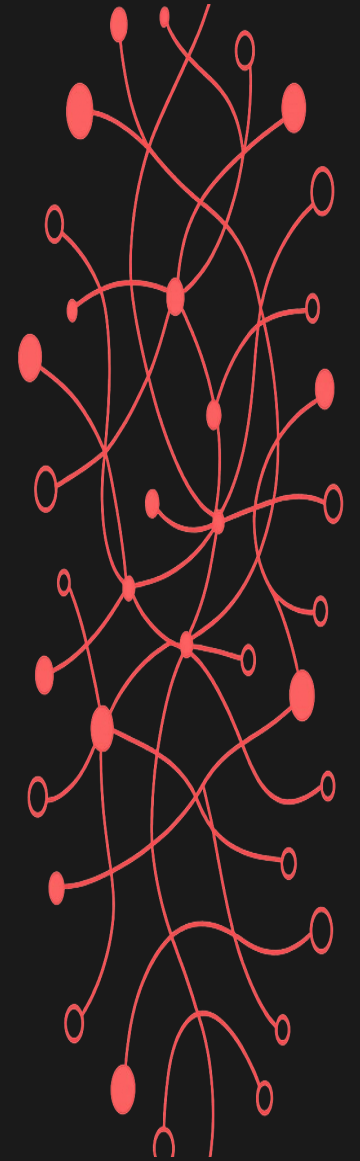
**Separar el agente
que hace el trabajo**



del agente que lo juzga
resulta una **palanca poderosa.**

03 Tres builds

el harness a prueba en proyectos reales



**Ambos agentes leen esto. El generator le apunta
antes de que el evaluador siquiera corra.**

Calidad de diseño

Un todo coherente, no una colección de partes

Originalidad

Decisiones propias, penaliza los defaults de template

Oficio

Tipografía, espaciado, contraste

Funcionalidad

¿Los usuarios completan tareas sin adivinar?

"Construí un creador de juegos retro 2D"

	Un solo agente	Harness completo
Duración	20 min	6 hs
Costo	\$9	\$200
Gameplay principal	Roto	Funciona
Features	Mínimas	16 features, 10 sprints
Integración de arte	Ninguna	Generador de sprites integrado

El run de \$9 vs el de \$200

\$9

Un agente, 20 minutos

- Reporta éxito
- Se ve pulido
- Gameplay principal roto

\$200

Harness completo, 6 horas

- 10 sprints, calificados
- 16 features entregadas
- Gameplay principal funciona

22x el costo. El producto que de verdad funciona.

La separación está en la config, no en el prompt

```
generator.md

---
tools: Read, Write, Edit, Bash, Glob, Grep
---

Construye. No testea.
```

generator.md

```
evaluator.md

---
disallowedTools: Write, Edit, NotebookEdit
---

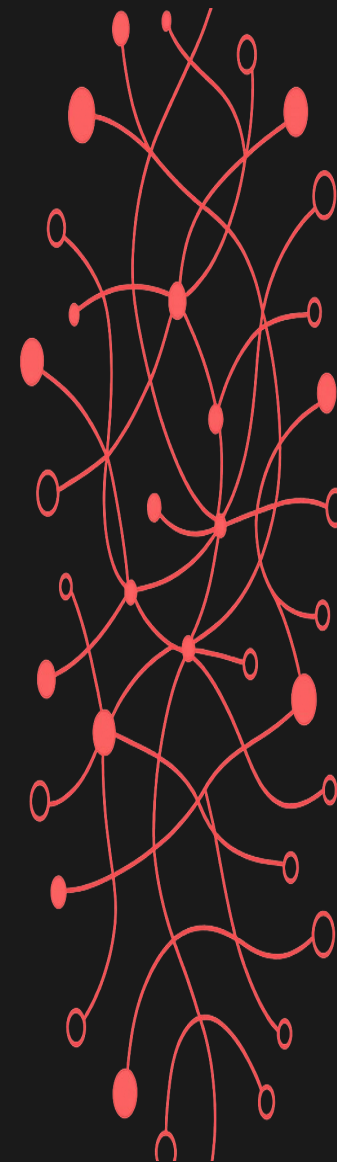
Testea. No puede editar.
```

evaluator.md

Mismo modelo. Distintos permisos. Incentivos asimétricos.

04 El harness es temporal

lo que hoy es andamiaje, mañana es entrenamiento



Cada versión del modelo achica el harness

V1 (Sonnet 4.5)

Planner

Generator

Evaluator

~~Sprints~~

~~Reseteos de contexto~~

V2 (Opus 4.6)

Planner

Generator

Evaluator

Los sprints y los reseteos de contexto desaparecieron cuando el modelo solo ya sostenía el hilo.

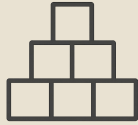
El harness de hoy es el dato de entrenamiento de mañana.

Cada componente de un harness
codifica una suposición sobre lo que el
modelo no puede hacer solo. Esas
suposiciones vale la pena **estresarlas**.





**Separar
Generator/Evaluator**



**Criterios
concretos**



**QA
adversarial**



**Re-evaluar
cada release**

Armá el tuyo

- Dos configs, dos roles
- Criterios verificables, no vibes
- El juez es el techo de calidad
- Retirá piezas del harness a medida que el modelo mejora
- <https://github.com/ddakaszewicz/generator-evaluator-demo>

el patrón sobrevive aunque cambie el modelo